

# MCUNet Reproduction: Tiny Deep Learning on IoT Devices

1<sup>st</sup> Devin Setiawan  
*Electrical Engineering and  
Computer Science  
University of Kansas  
Lawrence, KS, USA  
devinryandi.s@ku.edu*

2<sup>nd</sup> Vinayak Jha  
*Electrical Engineering and  
Computer Science  
University of Kansas  
Lawrence, KS, USA  
vinayakjha@ku.edu*

**Abstract**—Edge AI applications often require deploying deep learning models on microcontrollers with extremely limited memory and compute resources. MCUNet, a state-of-the-art framework, combines TinyNAS for neural architecture search and a memory-efficient inference engine tailored for such constrained environments. In this project, we reproduce the MCUNet results and implement the TinyNAS search strategy to automatically design lightweight convolutional neural networks optimized for microcontrollers. Using the Rock Paper Scissor dataset, we train and evaluate models under strict resource constraints. Our implementation achieves small accuracy trade-offs while maintaining a small model footprint, validating the reproducibility and practical utility of the MCUNet framework. Furthermore, we demonstrate the robustness of the TinyNAS methodology by experimenting with different design choices, such as replacing the lightweight backbone with ResNet variants and optimizing models using TensorFlow Lite instead of the original TinyEngine. These variations show that TinyNAS can generalize well across different deployment and architectural settings while still meeting tight resource budgets. Challenges in replicating the training pipeline, adapting search parameters, and deploying to embedded platforms are also discussed.

**Keywords**—*TinyNAS, MCUNet, IoT, Neural Architecture Search*

## I. INTRODUCTION

Deep learning has found applications in all levels of the technology stack, from centralized servers to edge devices. Edge AI applications benefit from reduced latency and allow off-the-network embedded deployments, thus providing significant benefits over centralized server deployments. However, edge AI deployment has unique challenges, such as limited working memory size and compute resources.

MCUNet [1] is a system-model co-design framework for deep learning on off-the-shelf microcontrollers. In this work, we have the following contributions:

1. Deployment of MCUNet for Visual Wake Words (person or no-person detection) task, and reproduce the accuracy reported for this task in the original paper [CIT]

2. Extend TinyNAS to utilize TensorFlow Lite (TFLite) [2] as the backend instead of the original TinyEngine.

The paper is organized as follows. Section II presents Related Works. Section III presents the Methodology, where we delineate our extension of TinyNAS and deployment strategy on the STM32F746 microcontroller. Next, in Section IV, we present the experimental results and, in Section V, provide a corresponding discussion. Finally, we present the conclusion in Section VI.

## II. RELATED WORKS

### A. MCUNet

MCUNet [1] is a framework designed to enable deep learning, specifically tiny deep learning on Internet of Things (IoT) devices based on microcontroller units (MCU). Microcontrollers present a significant challenge for deep learning deployment due to their extremely limited memory, which is orders of magnitude smaller than mobile phones or cloud GPUs. To address this, MCUNet adopts a system-algorithm co-design approach that jointly optimizes the neural network architecture and the inference library to fit the tight memory and storage constraints. MCUNet consists of two components, TinyNAS (for neural architecture search) and TinyEngine (for deployment / inference). In this work, we focus on reimplementing and extending TinyNAS.

### B. TensorFlow Lite (TFLite)

TensorFlow Lite [2] is high-performance interpreter-based runtime for deploying deep learning models on edge AI devices. TFLite provides optimization strategies such as utilizing a single statically allocated memory for intermediate computations and buffers of a neural network. In this work, TFLite is utilized in the TinyNAS extension.

## III. METHODOLOGY

This section outlines the approach we used to reproduce the MCUNet framework and implement TinyNAS for deployment on resource-constrained microcontrollers. Our methodology is structured to mirror the original MCUNet paper while incorporating several alternative design choices to

evaluate the flexibility and robustness of the TinyNAS search process. The following subsections detail the environment setup, dataset preparation, our tinyNAS implementation, and our deployment workflow. We also highlight the rationale behind using different components such as TensorFlow Lite in place of TinyEngine and ResNet-based backbones instead of MobileNet-style ones to explore the generalizability of the TinyNAS methodology.

#### A. Overview of MCUNet and TinyNAS

One key component of MCUNet is TinyNAS, a two-stage neural architecture search (NAS) method. TinyNAS is responsible for designing efficient neural architectures suitable for these constrained environments. Its two stages involve first optimizing the search space to fit the resource constraints of the specific device, and then specializing the network architecture within that optimized space. This automated search space optimization considers diverse constraints (device, latency, energy, memory) with low search costs. By working in conjunction with the memory-efficient inference engine, TinyEngine, TinyNAS can explore a larger design space and find models with higher accuracy that can still run within the tight memory budget of microcontrollers. This co-design framework was the first to achieve over 70% ImageNet top-1 accuracy on a commercial microcontroller.

#### B. Environment Setup

Our experiments were conducted across two main environments: a local Ubuntu workstation for deployment and embedded development, and Google Colab for neural architecture search using TinyNAS. On the local environment, we used Ubuntu 24.04 LTS, the latest stable release at the time of experimentation. For Python-based development, two different versions were used to match the requirements of different toolchains: Python 3.11.12 was used for running TinyNAS code in Google Colab, while Python 3.6 was used locally to ensure compatibility with certain embedded deployment packages.

We implemented TinyNAS using TensorFlow v2.18.0, with training and evaluation executed on Google Colab, which provided access to GPU acceleration. Model architecture search and export were handled entirely within this Colab environment. For deployment to microcontroller hardware, we used STM32CubeIDE version 1.6.1. Although the original MCUNet implementation recommends version 1.5.0, it was not publicly available at the time of this project. Our deployment targeted the STM32F746 development board paired with an Arducam camera module, enabling inference on real image input in embedded settings.

#### C. Dataset Preparation

To evaluate the performance of pre-trained MCUNet models on visual wake word detection, we used the small Visual Wake Words (VWW) dataset provided in the official MCUNet GitHub repository. We preserved the original data pipeline without any modifications to ensure consistent comparison with the reference implementation.

For our TinyNAS training and evaluation, we diverged from the original MCUNet paper and instead used the Rock-Paper-Scissors dataset available in the TensorFlow

Datasets (TFDS) library. This dataset was chosen for its simplicity and ease of integration, allowing us to focus on testing the flexibility and robustness of the TinyNAS search strategy.

We split the Rock-Paper-Scissor dataset as follow:

- 80% of the training set was used for training,
- 20% for validation,
- and the entire test set for final evaluation.

#### D. TinyNAS Search Process

The TinyNAS methodology implemented in this work follows a four-stage pipeline consisting of search space optimization, supernet training, architecture search, and final architecture evaluation. This pipeline enables efficient neural architecture search tailored to edge-device constraints.

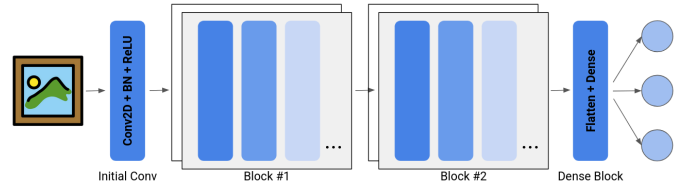


Fig. 1. Resnet-like model architecture as search space instead of MobileNet-like architecture used in the original TinyNAS.

1) Search Space Optimization: Our search space consists of input resolutions  $R=[32,64,96,128]$ , and width  $W=[16,32,48,64]$ . Each subspace was evaluated under a strict resource constraint of less than 500 KB model size using TFLite for profiling. We selected the subspace with the highest average FLOPs as it provided a better trade-off between model complexity and deployment feasibility. Within this subspace, we set  $M = 200$ , meaning 200 candidate models were analyzed to guide the search.

2) Supernet Training: We trained a supernet that contains all sub-network in a given search space. The supernet was trained for 50 epochs on the Rock-Paper-Scissors dataset using weight sharing, allowing all subnetworks to efficiently co-adapt their parameters during training.

3) Architecture Search: An evolutionary algorithm was employed to explore the selected search space. The search maintained a population of 10 subpopulations, each containing 10 candidate architectures. In each iteration, candidates are sub-network obtained from the supernet that were evaluated based on their fitness, defined by test set accuracy. We performed a lightweight retraining for 5 epochs, in contrast to the 50 epochs used for the original supernet to align the sub-network.

4) Final Architecture Evaluation: The best-performing architecture discovered during the search was instantiated and retrained independently of the supernet. We retrain it for 50 epochs on the complete training set and evaluate it on the testing set.

### E. Deployment to Microcontroller

Deployment to the STM32F746 IDE was performed using the STM32CubeIDE. For this paper, static images were utilized. The images were chosen using compile-time options, and switching images required recompiling the executable and reprogramming the microcontroller.

## IV. EXPERIMENTS AND RESULTS

This section presents the validation of the pre-trained models from the MCUNet Model Zoo on the Visual Wake Words (VWW) dataset, confirming their reported performance and highlighting the trade-off between accuracy and inference time. We also describe our efforts in deploying a selected model to the STM32F746 microcontroller. Finally, we present the results of our TinyNAS pipeline on the Rock-Paper-Scissors dataset, comparing the supernet and the discovered best-fit architecture in terms of accuracy, size, and inference latency under a strict resource constraint.

### A. Model Zoo Evaluation

TABLE I. ACCURACIES OF VWW MODELS

| Model       | Reported | Actual |
|-------------|----------|--------|
| mcunet-vww0 | 87.3%    | 87.3%  |
| mcunet-vww1 | 88.9%    | 88.9%  |
| mcunet-vww2 | 91.8%    | 91.6%  |

Fig. 2. The figure above represents the accuracies of the available model on the VWW dataset. The “Reported” are accuracies of each model as reported in the github. The “Actual” are accuracies that we got from running it on our machine.

We successfully replicated the classification accuracies reported for all Visual Wake Words (VWW) models provided in the MCUNet Model Zoo, confirming the reproducibility of the original results. To further analyze performance trade-offs, we profiled each model’s inference time and constructed a graph (Fig. 3) showing normalized inference latency across models. As expected, larger models exhibited proportionally higher inference times, illustrating the trade-off between model complexity and execution efficiency on constrained hardware.

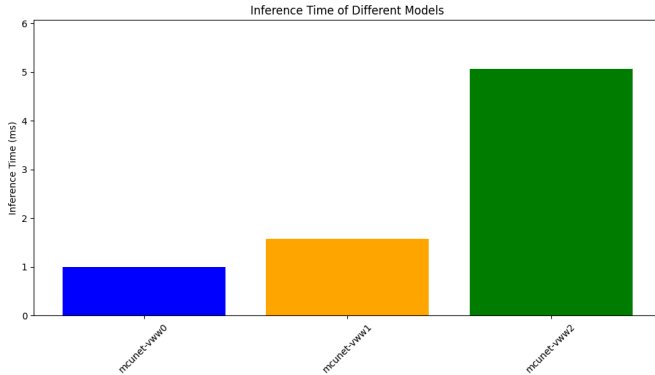


Fig. 3. The figure above represents the normalized inference time for different visual wake word models. The model size goes from smallest to largest (left to right). The result shows that as model size increase, inference time increase

### B. STM32 Edge Deployment

We successfully deployed the selected model onto the STM32F746 microcontroller, demonstrating its ability to perform inference under strict resource constraints. The results of the deployment are illustrated in Fig. 4 and Fig. 5, showing detection outputs for a negative and a positive sample, respectively. The background turning yellow serves as a visual indicator of a successful detection. However, due to issues with the connected camera module, the frames-per-second (FPS) counter did not function, preventing quantitative measurement of real-time inference performance.

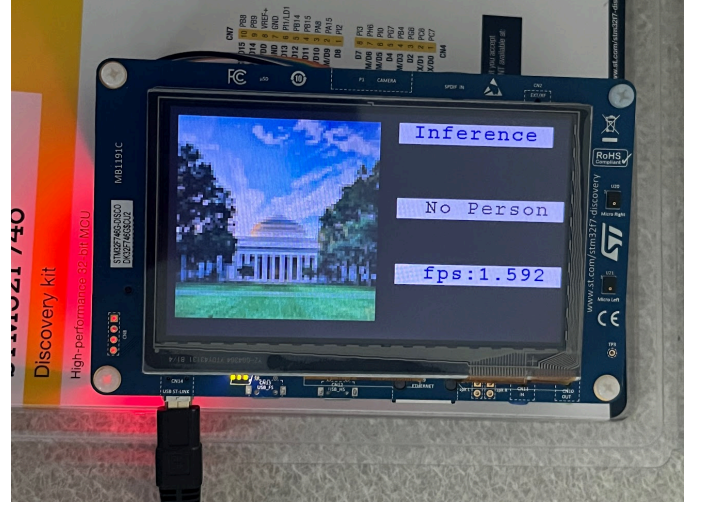


Fig. 4. Inference result on STM32F746 showing a negative sample. The background remains unchanged, indicating no detection event.



Fig. 5. Inference result on STM32F746 showing a positive sample. The background turns yellow, signaling successful detection by the deployed model.

### C. TinyNAS Results

We evaluated our TinyNAS implementation across multiple stages of the search pipeline. Fig. 6 presents the outcome of the search space optimization, where the configuration with input resolution 128 and width multiplier 64 yielded 61 out of 200 models that successfully converted to TFLite and met the <500KB hardware constraint. This search

space also achieved the highest average computational complexity at 1702 MFLOPs, making it the most expressive candidate. Following this, Fig. 7 displays the result of the evolutionary architecture search, where the best-performing sub-network emerged in generation 4, achieving a fitness score (validation accuracy) of 0.988. The selected architecture includes a  $7\times 7$  kernel at the input, with two  $5\times 5$  kernels in both block#1 and block#2. Finally, Fig. 8 compares the performance of this best sub-network against the supernet. While achieving 90% of the supernet’s normalized accuracy, the sub-network significantly reduces normalized inference time to 0.23 and model size to 0.18, making it suitable for our target hardware, unlike the full supernet which exceeds the memory constraint.

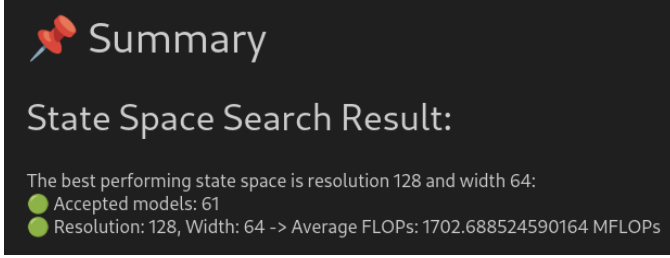


Fig. 6. Result of search space optimization from TinyNAS. Among the evaluated spaces, the configuration with resolution 128 and width 64 produced 61 out of 200 models that satisfied the <500KB constraint post-TFLite conversion. This space also achieved the highest average FLOPs (1702 MFLOPs), making it the most suitable candidate for further search.

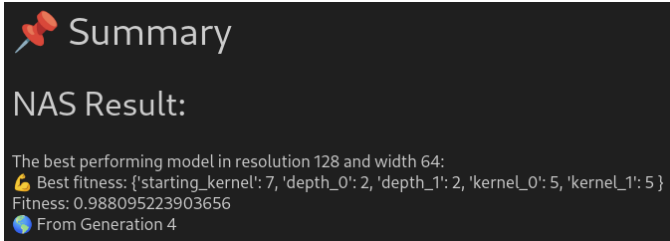


Fig. 7. Output of the evolutionary NAS search process. The best sub-network was discovered in generation 4, with a fitness (validation accuracy) of 0.988. The architecture includes a  $7\times 7$  input kernel, and two stages (depth\_0 and depth\_1) each with  $5\times 5$  kernels and depth of 2.

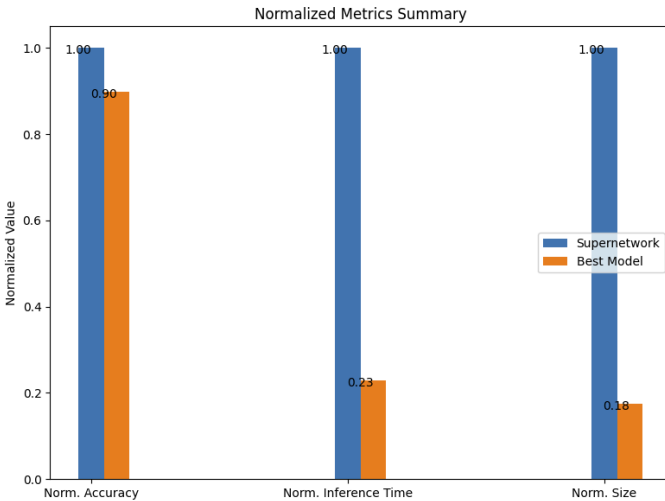


Fig. 8. Comparison of normalized accuracy, inference time, and model size between the supernet and the best sub-network selected by TinyNAS. The sub-network maintains 90% of the supernet’s accuracy while achieving significantly lower inference time (0.23 $\times$ ) and model size (0.18 $\times$ ), making it suitable for resource-constrained deployment.

## V. DISCUSSION

This section reflects on the broader implications of our implementation and experimental results. While our work successfully reproduces key findings from the MCUNet and TinyNAS pipelines, it also surfaces several important considerations around practical deployment, reproducibility, and design trade-offs. We elaborate on the challenges encountered during implementation, insights gained from reproducing results, and discuss inherent limitations and compromises made throughout our workflow. We also compare our findings with the original paper where possible.

### A. Implementation Challenges

A key challenge during the implementation was that the Arducam camera module did not function properly. The current leading hypothesis is that the old version of STM32CubeIDE (1.5.0) needs to be used for proper functioning. Running the inference under the debug mode reveals that the camera reads are blocking, and thus, the inference does not execute, leading to an empty screen

The proposed TinyNAS extension was computationally expensive during the search space optimization and led to high compute and memory resource usage, leading to rate limits on Google Colab. The network architecture search however, is very efficient and consumes a reasonable amount of GPU power.

### B. Insights from Reproduction

Our experiments revealed that TinyNAS is not tightly bound to specific architectural backbones or toolchains. While the original paper uses a MobileNet-like backbone in conjunction with TinyEngine, we demonstrate that the same methodology is applicable to ResNet-like architectures. This suggests that as long as a weight-sharing strategy can be clearly defined, the core optimization process remains effective across a wide range of backbone types. Moreover, our implementation highlights that the methodology can potentially extend to more advanced models. For instance, since our deployment backend uses TensorFlow Lite (TFLite), which supports Transformer operations, the TinyNAS pipeline could in theory be adapted to Transformer-based models by defining appropriate weight-sharing mechanisms for Transformer layers.

We also observed that several key parameters in the TinyNAS pipeline can be modified without breaking the methodology. One of the most impactful changes we made was redefining the hardware constraint: whereas the original paper optimizes for peak memory usage (RAM), we optimized for flash storage size. This shift demonstrates that any resource constraint such as inference latency or power consumption that is measurable after model conversion can serve as a valid optimization target. Also, the neural architecture search (NAS) process, particularly the evolutionary algorithm used to discover optimal subnetworks,

is highly customizable. There are no hard requirements for population size, initialization strategies, fitness functions, or selection policies. This flexibility opens the door to a variety of NAS configurations tailored to specific deployment environments and optimization goals.

### C. Limitations of TinyNAS

While TinyNAS provides a flexible and effective framework for neural architecture search under resource constraints, our reproduction also revealed notable limitations. One major bottleneck lies in the search space optimization phase, which, although relatively lightweight compared to full architecture search, remains both resource-intensive and time-consuming. In our experiments, evaluating a single search space required approximately 20 minutes due to the overhead of converting each candidate model to TensorFlow Lite (TFLite) format. This TFLite conversion step, necessary to assess whether a model satisfies the hardware constraints, becomes the dominant cost and presents a scalability challenge. Without access to a significantly faster model converter or simulator, the practicality of using TinyNAS for larger or more complex search spaces becomes increasingly limited.

Another limitation arises from the heuristic selection of the optimal search space based on average FLOPs (floating point operations). While we observed that the search space with the highest average FLOPs yielded the best balance between accuracy and feasibility in our task, this correlation lacks formal justification. It remains unclear whether this heuristic generalizes across other machine learning tasks, architectures, or datasets. Relying on average FLOPs as a proxy for representational power may oversimplify the complex interactions between architecture, task-specific features, and hardware performance. As such, this criterion warrants deeper theoretical and empirical investigation before being adopted as a general principle in future applications of TinyNAS.

## VI. CONCLUSION

We successfully deployed MCUNet for static visual wake words (person or no-person detection) on the STM32F746. We also presented the current implementation challenges of utilizing Arducam for camera-based inference tasks. Finally, we extend the TinyNAS to utilize TFLite instead of TinyEngine to reduce time required in the neural architecture search stage. In the future work, we will utilize Arducam for real-time person detection through a deeper investigation of the highlighted issues, and their subsequent resolution, in the STM32CubeIDE.

## ACKNOWLEDGMENT

We would like to thank Dr. Heechul Yun for his guidance and support throughout the course of this project, as well as for providing access to the STM32 development board used in the deployment phase. We also acknowledge the authors of MCUNet for open-sourcing their work, which served as the foundation for our reproduction and extension efforts.

## REFERENCES

- [1] Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. MCUNet: tiny deep learning on IoT devices. In Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 982, 11711–11722.
- [2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: a system for large-scale machine learning. In Proceedings of the 12th USENIX conference on Operating Systems Design and Implementation (OSDI'16). USENIX Association, USA, 265–283.